Spotter AI | Full Stack Developer Assessment

Weather-Aware Truck Routing

Project report

Prepared by: Ameema Rashid

Submitted: 3 November 2026

Stack: React, TypeScript, Material UI, MapLibre (frontend); Python, Django, Django REST Framework (backend)

Deliverable	Link
Hosted application	https://spotter-routing-ameema.vercel.app
Backend API (health check)	https://spotter-routing-api.vercel.app/api/health/
GitHub repository	https://github.com/ameema28/spotter-routing
Loom walkthrough	https://www.loom.com/share/dd06cacf76684728bac43abc59615a0a

SECTION 1

Overview

A long truck trip can take a full day or more. The weather a driver sees when leaving Dallas is not the weather they will meet near Memphis twelve hours later, and a heavy load makes strong wind more dangerous. This application helps a driver or dispatcher compare three routes and choose the safest one, based on the weather each truck will meet at each point of the trip, at the time it actually gets there.

Inputs and outputs

The user enters an origin, a destination, a departure date and time, and the load weight in pounds, and chooses how often the weather is checked along the way (every 10, 25 or 50 miles). The application returns:

- three route options drawn on a map, with the recommended route highlighted and a one-line reason that names the rule which decided it;
- a checkpoint at every sampling interval, coloured by risk level at the truck's estimated arrival time, with a popup showing the ETA, the weather values and why it received that level;
- a summary card per route: distance, driving time, total trip time including legally required breaks, and the number of miles at each risk level;
- a weather heatmap over the whole trip corridor with a 0 to 48 hour forecast slider, plus markers showing where each truck is at the selected hour.

Who it is for

The interface is aimed at a dispatcher or driver planning a trip before departure. Quick-fill buttons for three test trips (Dallas to Chicago, Denver to Salt Lake City, Kansas City to Denver) make the app easy to try. The matched place names are always shown, so a misspelt city is visible immediately.

SECTION 2

System Design and Architecture

The project has two parts, each deployed on Vercel's free tier. The React frontend handles everything the user sees. The Django backend does all the routing, weather and risk work and exposes a small REST API.

Layer	Technology	Reason
Frontend	React, TypeScript, Material UI, MapLibre GL with the OpenFreeMap basemap	React and Material UI are required by the role. MapLibre draws on the GPU, which keeps the heatmap slider smooth, and the basemap needs no API key in the browser.
Backend	Python, Django, Django REST Framework	Required by the role. The core rules are plain Python modules with no Django imports, so they are simple to test and to read.
Routing	Mapbox Directions and Geocoding	Real long-haul routes with distance and duration for every road segment.
Weather	Open-Meteo hourly forecast	Many locations per request, returned directly in mph and inches per hour.
Hosting	Vercel (both parts)	Free, no card, no fixed sleep, and a 300-second request limit.

Request flow

A search is a single call to `POST /api/route-plan/`. The backend geocodes both places, requests the routes and the extra route candidates in parallel, adjusts the timings for a truck, places the checkpoints, fetches all the weather in batches, scores every checkpoint and route, ranks the routes and builds the heatmap. Everything is returned in one response of about 11 KB (compressed), so moving the slider never needs another request.

Reliability and limits

- Forecasts are cached for 30 minutes per point, snapped to a 0.02-degree grid.
- Each client can search 10 times a minute and 100 times a day, which protects the free weather quota.
- If the weather service is busy, the routes still load and the heatmap retries itself after a minute through `POST /api/heatmap/`, fetching only the missing cells.
- Every error is returned with a clear message and the right status code (400, 422, 429, 502, 503, 504).

SECTION 3

Core Logic: Routes, Weather, Risk and Recommendation

All of this lives in `backend/routing/domain/` and was written test-first.

Three routes

Mapbox returned at most two routes in every test, so the third route is generated. The app also asks for a toll-free route and routes through major interstate hub cities on either side of the trip. A candidate is kept only if it is at most 25% slower than the fastest route and shares at most 75% of its road with the others; the most distinct one wins (random offsets sent trucks onto mining roads in testing).

Truck arrival times

Mapbox times are car times, so speed is capped at 65 mph and the US federal hours-of-service rules are added: a 30-minute break after 8 hours of driving and a 10-hour rest after 11 hours (FMCSA summary and 49 CFR 395.3). Dallas to Chicago changes from about 15 hours of driving to a 26-hour trip, which changes the forecast hour used at most checkpoints.

Checkpoints and weather

A checkpoint is placed every 10, 25 or 50 miles plus the destination, each with its own ETA. Each checkpoint represents the road halfway to its neighbours, so "Severe miles" add up to the real route length. Rain is *rain + showers*, snow is *snowfall* and wind is the sustained 10 m wind speed. Because rain and snow are totals for the hour ending at a timestamp, they use the hour ending at or after the ETA; wind is an instant reading, so it uses the nearest hour.

Risk levels and load rules

Each value is placed in the brief's bands. Where the table repeats an edge (for example 0.25 in/hr), the riskier band is used; rain of exactly 1.00 in/hr stays Severe because No Travel is strictly above it. The checkpoint takes the worst of wind, rain and snow, and the load rules can only raise that level:

Wind	Load	Result
55 mph or more	Any	No Travel
45 to 54 mph	Over 30,000 lb	No Travel
35 to 44 mph	Over 40,000 lb	At least Severe

Ranking

Routes are first grouped into tiers: clean routes, routes with more than 25% unknown forecast miles, and routes with any No Travel point. Inside a tier the brief's order applies: fewest Severe miles, then fewest High miles, then lowest average risk, then shortest trip time. Only a clean route can be recommended; if none exists, the app says "No safe route, consider delaying departure" rather than guessing.

Heatmap

Up to 250 grid cells around the routes each carry 49 hourly levels (hour 0 is departure) built with the same rules and the user's load. The slider only changes which hour the colours read, so it stays instant.

SECTION 4

Testing, Performance and Quality

I wanted evidence for every claim, so each figure below comes from a test run, a live check or the QA log in `docs/QA.md`.

Measure	Result
Automated tests	465 backend tests (pytest) and 89 frontend tests (Vitest); core rules written test-first
Test strength	35 bugs deliberately injected into the logic one at a time, all caught; one missing case found and covered
QA pass	Requirements checklist, 4 live searches and 28 of 28 UI scenarios passing (rare outcomes, errors, form validation, keyboard, phone width)
Slider speed (live site)	About 2.6 ms of React work per move, drawn in the next frame (15.6 ms median)
Lighthouse (live, desktop)	Performance 89, Accessibility 100, Best Practices 100
Search time (live)	About 3.3 s for a new search, 2.1 s repeated, 7.7 s after a cold start
Heatmap accuracy check	Heatmap and checkpoint levels agree in 95.9% of cases (117 of 122); the rest differ by one level

A live example

During testing on 3 November, a storm over Arkansas and Missouri gave a clear example. The direct routes had Severe and No Travel checkpoints, including one with 1.008 in/hr of rain, just above the No Travel limit. The app recommended the generated route through Kansas City, which had no Severe or High miles, and ranked the fastest route last because it contained a No Travel point.

SECTION 5

Key Decisions and Trade-offs

The brief left several points open. Each choice and its reason is logged in `DECISIONS.md`; the most important ones are below.

Decision	Why
Shared table edges use the riskier band	It follows the brief's symbols and is the safer choice; every edge is tested.
Worst of wind, rain and snow; load rules only raise	Any single hazard can stop a truck, and heavy snow must never be downgraded by a wind rule.
Missing data is Unknown, never Low	A route that cannot be assessed should not look safe; a proven No Travel still counts.
Half-gap mile weights	Severe and High "miles" become real road miles that add up to the route length.
Hours-of-service breaks in the ETA	A single driver cannot legally drive a 969-mile trip without stopping, and the break changes the forecast hour.
No Travel routes ranked last; recommend none if nothing is safe	The brief calls No Travel a stop signal, so it is never traded against Severe miles.
Third route via hub cities	Random offsets produced unrealistic roads; hub cities give real, distinct alternatives.
Weather cache at 0.02 degrees	A coarser 0.1-degree grid changed storm risk levels in about half of the active hours.
Heatmap built once, slider changes only the colour rule	Keeps the slider smooth without any server calls.
Vercel hosting with a heatmap retry	No sleeping server for reviewers in other time zones, and a busy weather service costs a minute, not a whole search.

SECTION 6

Limitations and Future Work

The current version keeps to the brief's priority of correctness and clarity over extra features. These are the known limits and what I would do next:

- **Car-profile routing.** Mapbox has no truck profile, so height, weight and hazardous-material restrictions are not considered. A truck-specific routing API would be the next step.
- **Simplified hours of service.** The 60/70-hour weekly limits, sleeper-berth splits, adverse-condition extensions, team driving and real rest-stop locations are not modelled.
- **Weather provider limits.** The free Open-Meteo tier counts every location and Vercel shares outbound IP addresses, so very busy periods can hit the limit. A paid plan or a fixed-IP host would remove this.
- **Per-instance cache and throttling.** Both reset when the server restarts; a shared store such as Redis would fix this.
- **Place matching.** A misspelt place is matched silently (the matched name is shown). A "did you mean" step would make this clearer.
- **Phone layout.** On small screens the slider's hour labels are hidden and the time label wraps.

SECTION 7

Development Process, AI Use and Time

I worked in small, planned blocks over four days. Before each block I confirmed the plan and the decisions; after it, I checked the tests and the result in the browser, then committed. The core logic was written test-first, and every third-party fact (route counts, units, rate limits, legal break rules) was checked with a real call or an official source before the code relied on it.

Use of AI

I planned the work with Claude and built it with Claude Code, one block at a time in plan mode. I made every decision myself, reviewed each plan and the tests before code went in, and verified the results in the browser. The commit history shows this block-by-block process, with Claude listed as co-author.

Time spent

Day	Hours	Main outcome
Day 1	2.5	Planning, Django skeleton, real API checks, risk/load/sampling/ETA/ranking logic with tests
Day 2	3.25	API clients, caching, third route, truck ETA, route-plan endpoint, backend deployed
Day 3	2.75	React frontend: form, results, map, heatmap and slider, performance work, frontend deployed
Day 4	6.25	Heatmap retry, QA pass, final README, report and Loom walkthrough
Total	14.75 of 16	Within the 4-day and 16-hour limits